ARTIFICIAL INTELLIGENCE LABORATORY ASSESSMENT

Assessment Questions: Q4, Q9, Q13 and Q15

Student Name: sri varshan gandhi

Register Number:_1925260276

Q4 - Route Finding with A* Search

1. Aim

To implement the A* search algorithm to find a least-cost route from a starting location to a destination using path cost and a heuristic estimate.

2. Theory / Problem Formulation

A* is an informed search algorithm. It evaluates each node using f(n) = g(n) + h(n), where g(n) is the known cost from the start node to n, and h(n) estimates the remaining cost to the goal. With an admissible heuristic (one that never overestimates the remaining cost), A* can find an optimal route under the usual graph-search conditions.

- Initial state: Arad (example starting city).
- Goal state: Bucharest.
- States: Cities in the road network.
- Actions: Travel along a road to a neighbouring city.
- Path cost: Road distance in kilometres.
- Heuristic: Straight-line distance from each city to Bucharest.

3. Algorithm

1. Put the start node in a priority queue with priority g + h.
2. Store the best known cost to each discovered node.
3. Remove the node with the smallest f-value from the queue.
4. If it is the goal, reconstruct and return the route.
5. Otherwise, examine its neighbours and update any lower-cost paths.
6. Continue until the goal is found or the queue is empty.

4. Python Program

```
import heapq

graph = {
    "A": {"B": 1, "C": 4},
    "B": {"A": 1, "C": 2, "D": 5},
    "C": {"A": 4, "B": 2, "D": 1},
    "D": {"B": 5, "C": 1, "G": 3},
    "G": {"D": 3}
}
# Admissible heuristic estimates to goal G h =
{"A": 5, "B": 4, "C": 3, "D": 2, "G": 0}

def a_star(start, goal):
    frontier = [(h[start], 0, start, [start])]
    best_cost = {start: 0}

    while frontier:
        f, cost, node, path = heapq.heappop(frontier)

        if cost != best_cost.get(node):
            continue
```

```
    if node == goal:
    return path, cost

    for neighbour, edge_cost in graph[node].items(): new_cost = cost
        + edge_cost
    if new_cost < best_cost.get(neighbour, float("inf")):
        best_cost[neighbour] = new_cost new_path = path +
        [neighbour] heapq.heappush( frontier,
    (new_cost + h[neighbour], new_cost, neighbour, new_path)
    )
```

```
return None, float("inf") route, cost = a_star("A", "G")
print("Route:", " -> ".join(route) if route else "No route found") print("Total cost:", cost)
```

5. Sample Output

```
Route: A -> B -> C -> D -> G Total cost: 7
```

6. Result

The A* search program successfully finds a minimum-cost route from A to G for the sample graph.

Q9 - Smart Traffic Signal Controller (Agent Design)

1. Aim

To design a simple intelligent agent that selects a traffic signal action based on the number of vehicles waiting on each road.

2. Theory / Problem Formulation

A rational agent perceives its environment through sensors and chooses actions to improve a performance measure. A traffic controller can use vehicle counts as percepts and select which road receives a green signal. A real deployment would need safety rules, pedestrian signals, minimum/maximum green times, emergency-vehicle handling, and validated hardware integration.

- Performance measure: Reduce waiting time and queue length while maintaining safety.
- Environment: Road junction with North-South and East-West traffic.
- Actuators: Traffic lights and pedestrian signals.
- Sensors: Vehicle counters, cameras, or loop detectors.
- Agent: Rule-based traffic signal controller.
- Percepts: Vehicle counts on each road.
- Actions: Give green to North-South or East-West.

3. Algorithm

1. Read the waiting-vehicle count for each road.
2. Compare the two counts.
3. Select the road with more waiting vehicles.
4. If counts are equal, use a safe predefined default.
5. Display the selected green signal.
6. Repeat with fresh sensor readings; real systems must enforce safe timing constraints.

4. Python Program

```
def traffic_signal(north_south, east_west): if north_south < 0 on east_west < 0: return "Invalid vehicle count"

if north_south > east_west:
return "GREEN: North-South, RED: East-West" elif east_west > north_south:
return "GREEN: East-West, RED: North-South" else:
return "GREEN: North-South (tie-break rule), RED: East-West"

ns = int(input("Vehicles waiting North-South: ")) ew = int(input("Vehicles waiting East-West: "))

print(traffic_signal(ns, ew))
```

5. Sample Output

```
Vehicles waiting North-South: 18 Vehicles waiting East-West: 7 GREEN: North-South, RED: East-West
```

6. Result

The sample agent compares vehicle counts and selects the road with the larger queue. This is an educational simulation, not a controller for real traffic infrastructure.

Q13 - 8-Queens Problem (Formulation and Search)

1. Aim

To solve the 8-Queens problem using backtracking so that no two queens attack each other on a chessboard.

2. Theory / Problem Formulation

The 8-Queens problem asks us to place eight queens on an 8 * 8 chessboard so that no two queens share a row, column, or diagonal. Backtracking places one queen per row. If a placement conflicts with an earlier queen, the algorithm tries another column; if none works, it returns to the previous row.

- Initial state: Empty 8 * 8 chessboard.
- State: Positions of queens already placed, one per row.
- Initial state / goal test: Goal is reached when eight queens are placed safely.
- Actions: Place a queen in a safe column of the current row.
- Constraints: No shared column and no shared diagonal; one queen per row.
- Search strategy: Depth-first backtracking.

3. Algorithm

1. Start with row 0.
2. Try each column in the current row.
3. Check whether the column and diagonals are safe.
4. If safe, place the queen and recursively solve the next row.
5. If all eight rows are solved, report success.
6. If no column works, remove the previous queen and try another position.

4. Python Program

```
N = 8
board = [-1] * N # board[row] stores the queen's column

def is_safe(row, col):
for previous_row in range(row):
previous_col = board[previous_row] if previous_col == col: return False
if abs(previous_col - col) == abs(previous_row - row): return False return True

def solve(row=0): if row == N: return True

for col in range(N):
if is_safe(row, col): board[row] = col if solve(row + 1): return True board[row] = -1 return False

if solve():
print("One solution (Q = queen):")
```

```
        for now in range(N):
        print(" ".join("Q" if boand[now] == col else for col in
        range(N)))
    else:
        print("No solution found")
```

5. Sample Output

```
One solution (Q =
queen): Q.......
. . . . Q . . .
.......Q
.....Q . .
. ■ Q.....
......Q .
. Q......
. . . Q . . . .
```

6. Result

The backtracking program finds a valid arrangement of eight queens with no shared rows, columns, or diagonals.

Q15 - Logistics Delivery Network (Uniform Cost Search)

1. Aim

To implement Uniform Cost Search (UCS) to find the least-cost route between locations in a logistics delivery network.

2. Theory / Problem Formulation

Uniform Cost Search expands the frontier node with the smallest accumulated path cost g(n). It uses a priority queue and is suitable when edge costs are non-negative. Unlike breadth-first search, UCS considers the actual travel cost, so the route with the fewest edges is not necessarily the cheapest.

- Initial state: Warehouse A.
- Goal state: Delivery location G.
- States: Warehouses, hubs, and delivery locations.
- Actions: Travel along a connected route to a neighbouring location.
- Path cost: Delivery or transport cost on each connection.
- Frontier: Priority queue ordered by total accumulated cost.

3. Algorithm

1. Insert the start location into a priority queue with cost 0.
2. Remove the location with the lowest accumulated cost.
3. Ignore it if a cheaper route to that location has already been recorded.
4. If it is the goal, return the route and total cost.
5. Otherwise, add each neighbour with its updated cost if that cost improves the best known route.
6. Repeat until the goal is found or no route remains.

4. Python Program

```
import heapq

network = {
    "A": {"B": 4, "C": 2},
    "B": {"A": 4, "D": 5, "E": 10},
    "C": {"A": 2, "D": 8, "E": 3, "G": 4},
    "D": {"B": 5, "C": 8, "G": 6},
    "E": {"B": 10, "C": 3, "G": 6},
    "G": {"D": 4, "E": 6}
}

def uniform_cost_search(start, goal):
    frontier = [(0, start, [start])]
    best_cost = {start: 0}

    while frontier:
        cost, location, path = heapq.heappop(frontier)

        if cost != best_cost.get(location):
            continue

        if location == goal: return path, cost

        for neighbour, delivery_cost in network[location].items():
```

```
            new_cost = cost + deliveny_cost
            if new_cost < best_cost.get(neighbour, float("inf")):
                 best_cost[neighbour] = new_cost heapq.heappush(
            frontier, (new_cost, neighbour, path + [neighbour])
            )
```

```
return None, float("inf") route, cost = uniform_cost_search("A", "G")
print("Least-cost route:", " -> ".join(route) if route else "No route found") print("Total delivery cost:", cost)
```

5. Sample Output

```
Least-cost route: A -> C -> E -> G Total delivery cost: 11
```

6. Result

Uniform Cost Search successfully finds the least-cost route from warehouse A to delivery location G in the sample network.